Lab Report: Particle Filtering and Various Applications

ANANDARAJAH Rahul

8 mars 2026

1. Problem Context

The programs implemented for this lab are available as attachments in `Exercice1.py` and `Exercice2.py`. The entirety of the code for exercise 1 and the relevant parts of the code for exercise 2 are available in the appendix. The face tracking results are available in `suivi1` and `suivi2`.

We have seen that Kalman filtering has many disadvantages when applied in its extended version to non-linear problems. Let us now look at a new type of filter : particle filtering. The challenge remains the same : take the example of an aircraft whose trajectory we wish to observe. The aircraft has a real trajectory and an observed trajectory. The observed trajectory may differ from the real trajectory because the observation may be disturbed by camera noise, for example. The problem is therefore to determine the real trajectory from the observations. We can model the problem with the following dynamic stochastic model :

$$\begin{cases} \mathbf{x_{n+1}} = f_{n+1|n}(\mathbf{x_n}, \mathbf{u_{n+1}}) \\ \mathbf{y_{n+1}} = g_{n+1}(\mathbf{x_{n+1}}, \mathbf{v_{n+1}}) \end{cases}$$

In the case of Kalman filtering, $(\mathbf{x}, \mathbf{y})$ is a hidden Markov chain, $f_{n+1|n}$ and g_{n+1} are linear and $\mathbf{u_{n+1}}$ and $\mathbf{v_{n+1}}$ are Gaussian, but these are strong assumptions and they are not always valid. In the rest of the lab, let us place ourselves in the case where $f_{n+1|n}$, g_{n+1}, $\mathbf{u_{n+1}}$ and $\mathbf{v_{n+1}}$ are arbitrary and retain the assumption that $(\mathbf{x}, \mathbf{y})$ is a hidden Markov chain. The density of the joint law can then be written as :

$$p(\mathbf{x_{0:n+1}}, \mathbf{y_{0:n+1}}) = p(\mathbf{x_{0:n+1}})p(\mathbf{x_{0:n+1}}|\mathbf{y_{0:n+1}}) = p(\mathbf{x_0}) \prod_{i=0}^{n} f_{i+1|i}(\mathbf{x_{i+1}}|\mathbf{x_i}) \times \prod_{i=0}^{n+1} g_i(\mathbf{y_i}|\mathbf{x_i})$$

Then, for every instant $n \in \mathbb{N}^*$, we seek to give the best estimator of $\mathbf{x_n}$ given all observations from 0 to n, $\mathbf{y_{0:n}}$. *Recalling the course* : let $H = \mathcal{L}^2(\Omega, \mathcal{A}, P)$, the Hilbert space of random variables defined on $(\Omega, \mathcal{A}, P)$, centered, square-integrable, equipped with the inner product $(X, Y) = \mathbb{E}(XY)$. Finding the best estimate of $\mathbf{x_n}$ given $\mathbf{y_{0:n}}$ amounts to minimizing the distance between $\mathbf{x_n}$ and the space spanned by $\mathbf{y_{0:n}}$. This distance is $\mathbb{E}(\mathbf{x_n}|\mathbf{y_{0:n}})$ and corresponds to the orthogonal projection of $\mathbf{x_n}$ onto H. We will then seek to determine/approximate this quantity.

2. Some Proofs

1. Importance Sampling without Importance Weights

Let $x \mapsto \pi(x)$ be a probability distribution and f an arbitrary function. We wish to compute the integral I :

$$I = \int f(x)\pi(x)\, dx = \int f(x)\, \pi(dx)$$

f and π may have simple expressions, but I can be difficult to compute. In scientific computing (MAT 4101), we have seen Monte-Carlo methods for computing approximate empirical numerical values of

integrals. Let $N \in \mathbb{N}^*$, let $X_1, \dots, X_N$ be independent and identically distributed according to the law p. By the law of large numbers :

$$\frac{1}{N}\sum_{i=1}^{N} f(X_i) \xrightarrow[N\to\infty]{\text{ps}} \mathbb{E}_\pi(f(X))$$

I then becomes :

$$I = \int f(x)\pi(x)\,dx = \int f(x)\,\pi(dx) \approx \frac{1}{N}\sum_{i=1}^{N} f(X_i) \text{ with } X_1, \dots, X_N \text{ i.i.d. according to the law } \pi.$$

However, Python does not necessarily know how to simulate draws according to the law π ! We therefore introduce a law q called the *importance distribution* that we know how to simulate in the integral :

$$\begin{aligned} I &= \int f(x)\pi(x)\,dx = \int \frac{f(x)\pi(x)}{q(x)} \cdot q(x)\,dx = \mathbb{E}_q\left(\frac{f(X)\pi(X)}{q(X)}\right) \\ &\approx \frac{1}{N}\sum_{i=1}^{N} \frac{f(X_i)\pi(X_i)}{q(X_i)} \quad \text{with } X_1, \dots, X_N \text{ i.i.d. according to the law } q! \end{aligned}$$

2. Importance Sampling with Importance Weights

In practice, we only know π up to a constant. For example, in our case $p(\mathbf{x}|\mathbf{y}) \propto p(\mathbf{x})p(\mathbf{y}|\mathbf{x})$. Thus, for all x, $\pi(x) \propto \pi'(x)$ with π' known. There exists M such that $\pi(x) = M\pi'(x)$, hence $M = \frac{1}{\int \pi'(t)\,dt}$, therefore $p(x) = \frac{\pi'(x)}{\int \pi'(t)\,dt}$. The computation of I then becomes :

$$\begin{aligned} I &= \int f(x)\pi(x)\,dx = \int \frac{\pi'(x)}{\int \pi'(t)\,dt} f(x)\,dx = \int \frac{1}{\int \frac{\pi'(t)}{q(t)} q(t)\,dt} \frac{\pi'(x)f(x)}{q(x)} q(x)\,dx = \int \frac{1}{\mathbb{E}_q\left(\frac{\pi'(X)}{q(X)}\right)} \frac{\pi'(x)f(x)}{q(x)} q(x)\,dx \\ &= \mathbb{E}_q\left(\frac{\frac{\pi'(X)f(X)}{q(X)}}{\mathbb{E}_q\left(\frac{\pi'(X)}{q(X)}\right)}\right) \approx \frac{1}{N}\sum_{i=1}^{N} \frac{\frac{\pi'(X_i)f(X_i)}{q(X_i)}}{\frac{1}{N}\sum_{j=1}^{N} \frac{\pi'(X_j)}{q(X_j)}} = \sum_{i=1}^{N} \frac{\frac{\pi'(X_i)}{q(X_i)}}{\sum_{j=1}^{N} \frac{\pi'(X_j)}{q(X_j)}} f(X_i) = \sum_{i=1}^{N} \frac{\omega_i}{\sum_{j=1}^{N} \omega_j} f(X_i) \end{aligned}$$

$$\text{with : } \begin{cases} X_1, \dots, X_N \text{ i.i.d. according to the law } q \\ \dfrac{\omega_i}{\sum_{j=1}^{N} \omega_j} \text{ is the } i^{\text{th}} \text{ normalized importance weight for all } i \in [\![1, N]\!] \end{cases}$$

3. Particle Filtering

The general principle of a filter is to determine $p(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}})$ from $p(\mathbf{x_n}|\mathbf{y_{0:n}})$:

$$\begin{aligned} &p(\mathbf{x_n}|\mathbf{y_{0:n}}) \\ &\quad\downarrow \text{ prediction} \\ &p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) = \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n})p(d\mathbf{x_n}|\mathbf{y_{0:n}}) \qquad (1) \\ &\quad\downarrow \text{ update} \\ &p(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) = \frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}})p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}})}{\int g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}})p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}})\,d\mathbf{x_{n+1}}} \qquad (2) \end{aligned}$$

The idea of particle filtering is to replace $p(\mathbf{x_n}|\mathbf{y_{0:n}})$ with a discrete empirical approximation obtained with a counting measure and to propagate this approximation through time.

① **Prediction :**

$$p(\mathbf{x_n}|\mathbf{y_{0:n}}) \approx \hat{p}(\mathbf{x_n}|\mathbf{y_{0:n}}) = \sum_{i=1}^{N} \omega_n^i \delta_{\mathbf{x_n^i}}(d\mathbf{x_n}) \quad \text{with} \begin{cases} \mathbf{x_n^1}, \dots, \mathbf{x_n^N} \text{ particles, i.e. empirical values of } \mathbf{x_n} \\ \forall i \in [\![1, N]\!],\ \omega_n^i \ \approx p(\mathbf{x_n^i}|\mathbf{y_{0:n}}) \text{ weight of particle i} \end{cases}$$

Substituting this result into (1), we obtain :

$$\begin{aligned} p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) &= \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}) p(d\mathbf{x_n}|\mathbf{y_{0:n}}) \approx \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}) \hat{p}(d\mathbf{x_n}|\mathbf{y_{0:n}}) \\ &= \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}) \sum_{i=1}^{N} \omega_n^i \delta_{\mathbf{x_n^i}}(d\mathbf{x_n}) = \sum_{i=1}^{N} \omega_n^i \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}) \delta_{\mathbf{x_n^i}}(d\mathbf{x_n}) \\ &= \sum_{i=1}^{N} \omega_n^i f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n^i}) \end{aligned}$$

The sum thus obtained is the discrete version of integral (1). This formula is used to predict $\mathbf{x}_{n+1}$ from $\mathbf{y}_{0:n}$ if $\mathbf{y}_n$ is missing, for example.

② **Update :**

Similarly, we will approximate $p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) = \int f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}) p(\mathbf{x_n}|\mathbf{y_{0:n}})\, d\mathbf{x_n}$. Since this is a mixture of distributions, we will first draw a label and then the new particles according to $f_{n+1|n}$ conditionally on the drawn label on i.i.d. random variables. Then, by the law of large numbers/importance sampling, the unweighted mean of the samples gives us an approximation of the density of the mixture of distributions. For all $j \in [\![1, N]\!]$:
- We draw a label whose probability of being drawn is proportional to its respective weight :

$$A_n^j \sim p(A_n^j = k) = \omega_n^k \approx p(\mathbf{x_n^k}|\mathbf{y_{0:n}})$$

- We draw the new particles :

$$\mathbf{x_{n+1}} \sim f_{n+1|n}(\mathbf{x_{n+1}}|\mathbf{x_n}^{A_n^j})$$

- We approximate $p(d\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}})$ by importance sampling, ensuring that each draw has the same weight :

$$p(d\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) \approx \hat{p}(d\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) = \frac{1}{N} \sum_{i=1}^{N} \delta_{\mathbf{x_{n+1}^i}}(d\mathbf{x_{n+1}})$$

- We substitute $\hat{p}(d\mathbf{x_{n+1}}|\mathbf{y_{0:n}})$ into (2) and obtain :

$$\begin{aligned} p(d\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) &\approx \frac{\frac{1}{N} g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}}) \sum_{i=1}^{N} \delta_{\mathbf{x_{n+1}^i}}(d\mathbf{x_{n+1}})}{\frac{1}{N} \sum_{i=1}^{N} g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})} \\ &= \sum_{i=1}^{N} \underbrace{\boxed{\frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}{\sum_{i=1}^{N} g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}}}_{i^{\text{th}} \text{ normalized importance weight}} \delta_{\mathbf{x}_{n+1}^i}(d\mathbf{x_{n+1}}) \end{aligned}$$

③ **One iteration of the particle filtering algorithm from step n to step n+1 :**

Algorithm 1 Une itération de l'algorithme de filtrage particulaire de l'étape n à l'étape n+1

INPUT : $(\mathbf{x}_n^i, \omega_n^i)$ avec $i \in [\![1, N]\!]$

for $j = 1$ to N **do**
  Tirer $A_n^j \sim p(A_n^j = k) = \omega_n^k$ #Resampling
  Tirer $\mathbf{x}_{n+1}^j \sim f_{n+1|n}(\mathbf{x}_{n+1}|\mathbf{x}_n^{A_n^j})$ #Sampling
  $\omega_{n+1}^j = g_{n+1}(y_{n+1}|x_{n+1}^j)$ #Weighting
end for

for $j = 1$ to N **do**
  $\omega_{n+1}^j = \frac{\omega_{n+1}^j}{\sum_j \omega_{n+1}^j}$ #Normalisation
end for

OUTPUT : $(\mathbf{x}_{n+1}^j, \omega_{n+1}^j)$ avec $j \in [\![1, N]\!]$

④ **Version of the proof seen in the lab : estimation of $\mathbb{E}(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}})$ and highlighting of importance sampling** :

$$\begin{aligned}
\mathbb{E}(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) &= \int \mathbf{x_{n+1}} p(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) d\mathbf{x_{n+1}} = \int \frac{\mathbf{x_{n+1}} p(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}})}{p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}})} p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) d\mathbf{x_{n+1}} \\
&= \int \frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}})}{p(\mathbf{y_{n+1}}|\mathbf{x_{0:n}})} \mathbf{x_{n+1}} p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) d\mathbf{x_{n+1}} \text{ importance distribution} \\
&\approx \frac{1}{N} \sum_{i=1}^{N} \frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}{p(\mathbf{y_{n+1}}|\mathbf{x_{0:n+1}})} \mathbf{x_{n+1}^i} \quad \forall i \in [\![1, N]\!] \ \mathbf{x_{n+1}^i} \sim p(\mathbf{x_{n+1}}|\mathbf{y_{0:n}}) \\
&= \sum_{i=1}^{N} \omega_{n+1}^i \mathbf{x_{n+1}^i} \quad \text{with} \quad \omega_{n+1}^i = \frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}{p(\mathbf{y_{n+1}}|\mathbf{x_{0:n+1}})}
\end{aligned}$$

For each i belonging to the interval $[\![1, N]\!]$, we normalize ω_{n+1}^i :

$$\tilde{\omega}_{n+1}^i = \frac{\omega_{n+1}^i}{\sum_{i=1}^N \omega_{n+1}^i} = \frac{g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}{\sum_{i=1}^N g_{n+1}(\mathbf{y_{n+1}}|\mathbf{x_{n+1}^i})}$$

Finally :

$$\mathbb{E}(\mathbf{x_{n+1}}|\mathbf{y_{0:n+1}}) \approx \sum_{i=1}^{N} \tilde{\omega}_{n+1}^i \mathbf{x_{n+1}^i}$$

3. Kitagawa Model

Let us study the Kitagawa model. Using the notations from the problem statement as well as our own, the dynamic stochastic model studied is as follows :

$$\forall n \in \mathbb{N} : \begin{cases} x_{n+1} = 0.5x_n + 25\dfrac{x_n}{1+x_n^2} + 8\cos(1.2(n+1)) + u_{n+1} \\ y_{n+1} = \dfrac{x_{n+1}^2}{20} + v_{n+1} \end{cases} \quad \text{with :} \quad \begin{cases} u_{n+1} \sim \mathcal{N}(0, Q) \\ v_{n+1} \sim \mathcal{N}(0, R) \end{cases}$$

Therefore :

$$\forall n \in \mathbb{N} : \begin{cases} f_{n+1|n}(x_{n+1}|x_n) = \mathcal{N}(x_{n+1}, 0.5x_n + 25\dfrac{x_n}{1+x_n^2} + 8\cos(1.2(n+1)), Q) \\ g_{n+1}(y_{n+1}|x_{n+1}) = \mathcal{N}(y_{n+1}, \dfrac{x_{n+1}^2}{20}, R) \end{cases}$$

Deducing the hidden Markov chain from observations seems difficult due to the square in the conditional likelihood law, which removes information about the sign of x_n even though we have an observation. We nonetheless apply particle filtering to the hidden Markov chain (x, y). We initialize the algorithm parameters as requested in the problem statement. The code is available in the appendix.

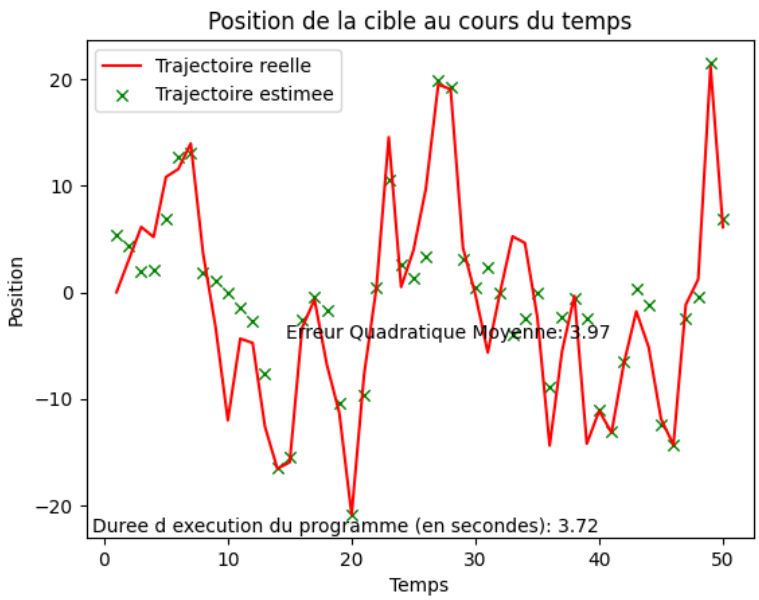

FIGURE 1 – Example of real and estimated target trajectories over time. The observed trajectory has been omitted to improve the readability of the curves.

Let us vary these parameters one by one to measure their impact on the algorithm's performance. To keep the report concise, instead of displaying curves, we will display performance metrics of the algorithm as a function of the parameter being studied.

1. Impact of N, the Number of Particles

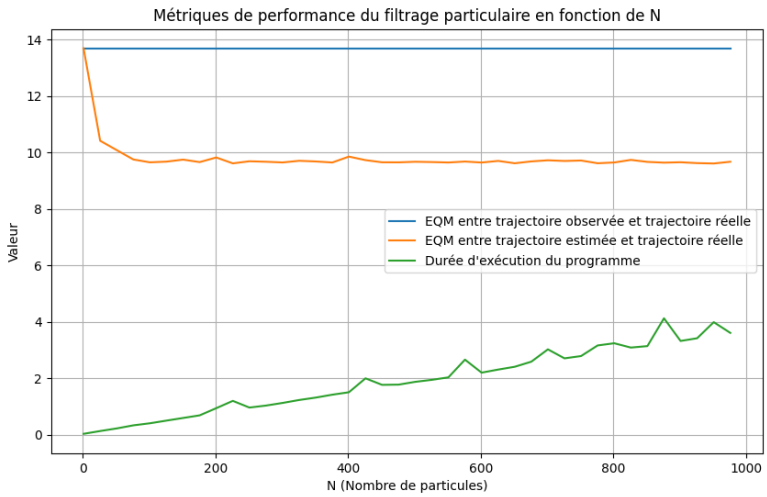

FIGURE 2 – We drew a real trajectory, an observation, and several estimated trajectories with numbers of particles ranging from 0 to 1000.

We can read from the curves that regardless of the number of particles, since the mean squared error (MSE) of the estimated trajectory is lower than that of the observed trajectory, the former is better than the latter. Furthermore, for a number of particles between 0 and 200, the MSE of the estimated trajectory decreases very rapidly as N increases. However, once N exceeds 200, there is no improvement in the estimated trajectory ; only the algorithm's execution time increases. The ideal number of particles is therefore 100 in this case. More generally, it can be useful to plot this curve to optimize the number of particles for a given problem, since using too many slows down the program without any added value in terms of MSE.

2. Impact of T, the Scenario Length

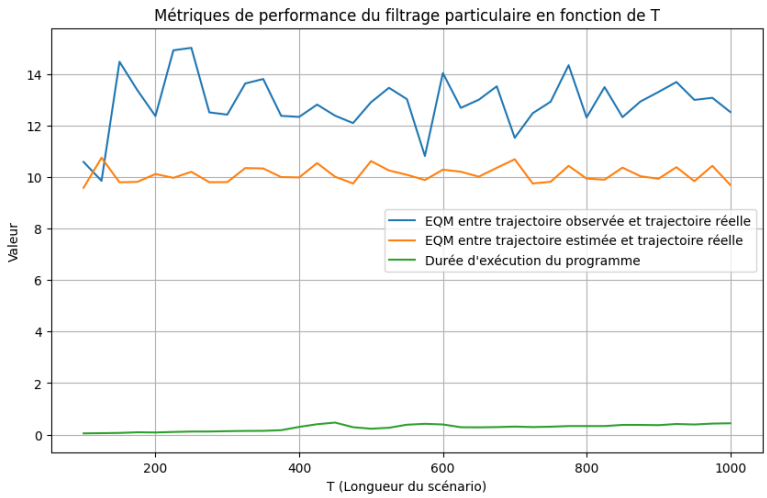

FIGURE 3 – We drew several real, observed, and estimated trajectories and displayed their MSE and particle filtering execution times as a function of T, the scenario length.

We can observe from the curves, once again, that particle filtering reduces the observation MSE. Regardless of the scenario length, both the MSE and the particle filtering execution times remain essentially the same. Since the principle of particle filtering consists of propagating particles with high weights, i.e. those that are strongly correlated with their observations, we expected the MSE of the

estimated trajectory to decrease as T increases, since the filter has more time to propagate the relevant particles.

3. Impact of Q, the Dynamic Noise

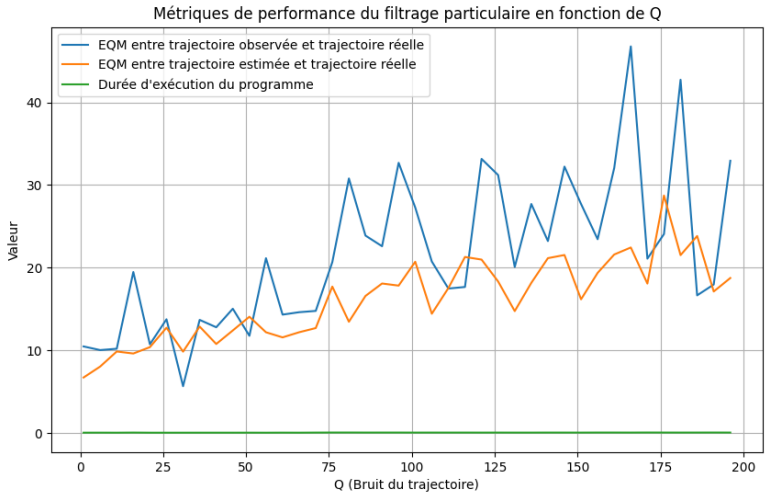

FIGURE 4 – We drew several real, observed, and estimated trajectories and displayed their MSE and particle filtering execution times as a function of Q, the dynamic noise.

The MSE of the estimated trajectory is generally lower than that of the observed trajectory, so the filter works : despite a noisy trajectory, good observations allow the observation MSE to be reduced. Nevertheless, both MSE values increase as Q increases. Particle filtering works but is very sensitive to dynamic noise and therefore performs increasingly poorly as we increase the dynamic noise.

4. Impact of R, the Observation Noise

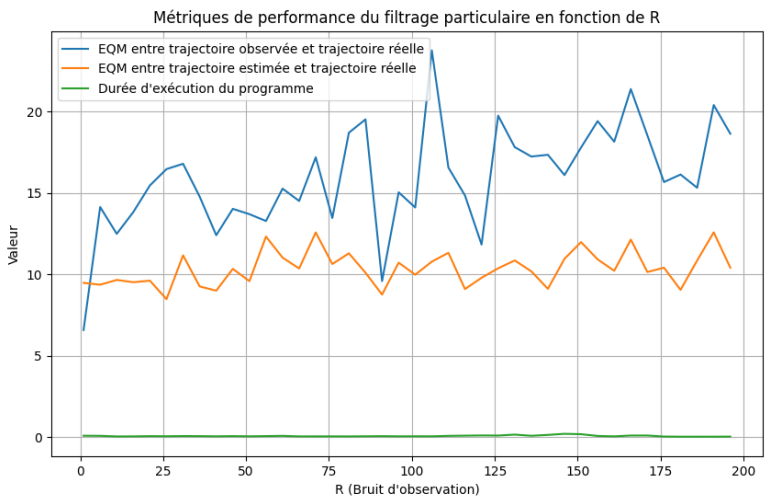

FIGURE 5 – We drew several real, observed, and estimated trajectories and displayed their MSE and particle filtering execution times as a function of R, the observation noise.

Regardless of the observation noise, the MSE of the estimated trajectory is lower than that of the observed trajectory : the filter works. The MSE of the observed trajectory increases with R but that of the estimated trajectory remains constant, around 10, and does not increase with R. Particle filtering appears to filter observation noise well, in contrast to dynamic noise.

4. Face Tracking on a Video Sequence

1. Context

We will now implement a particle filtering algorithm for face tracking. We have at our disposal a video of about sixty frames in which a student moves, and we wish to track their face. Using the notations from the problem statement as well as our own, we have the following dynamic stochastic model :

$$\forall n \in \mathbb{N}^* : \begin{cases} \mathbf{X}_n = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{X}_{n-1} + \mathbf{u}_n \\ g_n(\mathbf{Y}_n|\mathbf{X}_n) \propto \exp\{-\lambda D^2(q, q'(\mathbf{X}_n))\} \end{cases} \qquad \text{with :} \quad \begin{cases} \mathbf{u}_n \sim \mathcal{N}(0, \begin{bmatrix} C_1 & 0 \\ 0 & C_2 \end{bmatrix}) \\ D(q, q') = (1 - \sum_{n=0}^{N_b} \sqrt{q(n)q'(n)})^{\frac{1}{2}} \\ q \text{ the histogram of the first image} \\ q'(\mathbf{X}_n) \text{ the histogram of the region at position } \mathbf{X} \end{cases}$$

g_n is only known up to a constant, but this is more than sufficient since the likelihood only appears in the computation of the particles at the next step :

$$\forall n \in \mathbb{N}, \forall i \in [\![1, N]\!], \omega_{n+1}^i = \frac{g_{n+1}(\mathbf{y}_{n+1}|\mathbf{x}_{n+1}^i)}{\sum_{i=1}^N g_{n+1}(\mathbf{y}_{n+1}|\mathbf{x}_{n+1}^i)} = \frac{\exp\{-\lambda D^2(q, q'(\mathbf{X}_n^i))\}}{\sum_{i=1}^N \exp\{-\lambda D^2(q, q'(\mathbf{X}_n^i))\}}$$

The model we use is not truly adapted to reality. The problem statement assumes that the tracked object moves randomly at each instant, with a limited random distance, from step to step. However, the student does not always move step by step ; they may start slowly before accelerating. Although our model may not be perfect, as it does not capture these sudden movements, the observations we collect can help compensate for the errors caused by these unpredictable trajectories. However, in our previous study, we found that the filter was very sensitive to trajectory errors, which makes the situation somewhat difficult. One solution to filter dynamic noise would be either to increase the dynamic noise C_1 and C_2, which would allow for a larger radius during step-by-step movements ; or to film the student with a better camera with more frames per second, which amounts to artificially decreasing the acceleration.

Next, let us try to discuss the value of λ in the expression for the distance between two histograms. If λ is negative and very large in absolute value, the likelihood of all particles will also be very large. However, since the exponential function varies greatly, we will be able to easily select the relevant particles, given how different their respective weights would be. On the other hand, if λ is positive and very large, the likelihoods of all particles will be very small, making their propagation difficult because the exponential function will vary insignificantly. In this case, choosing the relevant particles will not be easy. Normally, we would intuitively choose λ as small as possible.

Let us nonetheless rewrite the particle filtering algorithm as proposed in the problem statement. The code can be found in the file `Exercice2.py`.

1. First Tracking, $\lambda = 20$

FIGURE 6 – Face tracking with particle filtering, $\lambda = 20$, frame 5

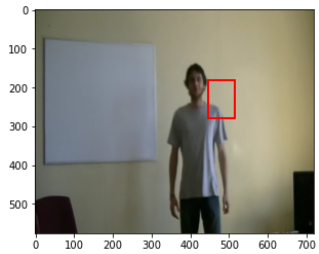

FIGURE 7 – Face tracking with particle filtering, $\lambda = 20$, frame 9

The assumptions are verified : tracking no longer works after too large an acceleration, as between frame 5 and frame 9. We get the impression that the algorithm searches for the face step by step and follows it. Once the filter loses the face, it no longer finds it and continues to search for it in the vicinity without ever finding it again. The tracking result can be found in the file `suivi1`.

1. Second Tracking, $\lambda = 70$

Let us take $\lambda = 75$. Our assumptions are verified : after too large an acceleration, the tracking loses the face, as during the transition from frame 5 to frame 9. On the other hand, it is able to find it again, as between frame 42 and frame 43. We observe an overall improvement in filter performance, which contradicts our initial predictions. Indeed, we observe that taking a higher positive value of λ seems to improve the filtering algorithm. However, we are unable to fully explain this phenomenon. The tracking result can be found in the file `suivi2`.

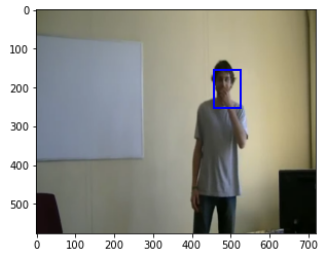

FIGURE 8 – Face tracking with particle filtering, $\lambda = 75$, frame 5

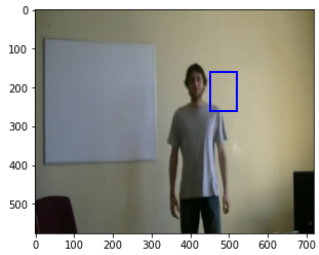

FIGURE 9 – Face tracking with particle filtering, $\lambda = 75$, frame 9

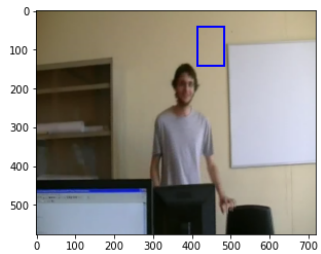

FIGURE 10 – Face tracking with particle filtering, $\lambda = 75$, frame 42

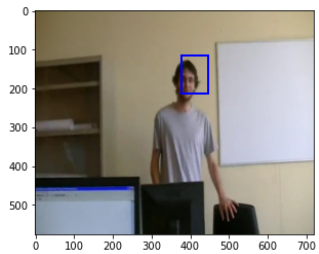

FIGURE 11 – Face tracking with particle filtering, $\lambda = 75$, frame 43

Let us try to explain why the algorithm becomes more performant by studying the effective number of particles N_{eff}.

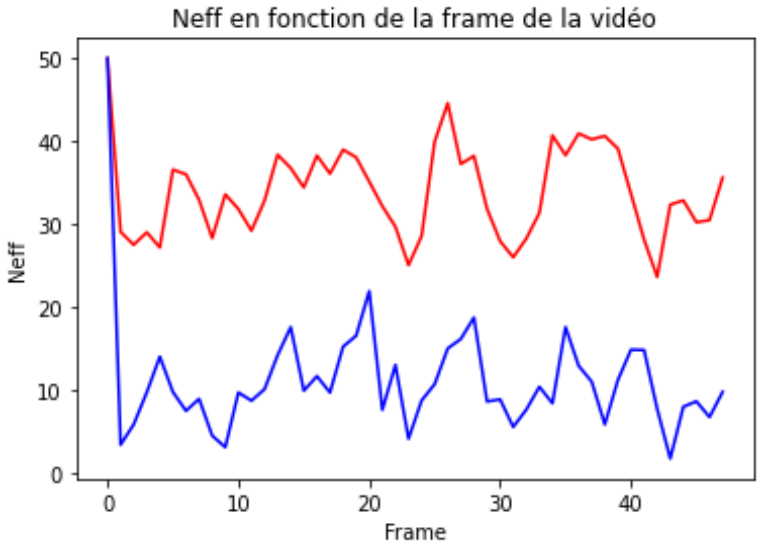

FIGURE 12 – We represented the effective number of particles as a function of the frame. The red curve represents the first tracking, the blue curve the second.

We notice that the number of particles in the second iteration of the algorithm (shown in blue), the one that performs best, has a lower average number of effective particles than in the first iteration of the algorithm (shown in red). We can interpret this by saying that the useless particles were not retained and that only the particles with the highest likelihood were kept. Frame-by-frame analysis of N_{eff} seems difficult.

We also implemented the solution proposed in the problem statement of increasing the dimension of the particles to account for zooms. Since compilation times were too long, we were unable to study it. The implementation of the solution can nonetheless be found at the end of `Exercice2.py`.

5. Appendix : Python Code

Exercise 1. Preamble

```
import numpy as np
import matplotlib.pyplot as plt

N = 50 # number of particles
Q = 10 # trajectory noise
R = 1 # observation noise
T = 50 # scenario length
```

Source 1 – Python code to initialize the problem parameters.

Exercise 1. Question 3

```
def creer_trajectoire(Q, x_init, T): # x_init is the initial position
    vecteur_x = np.zeros(T) # initialization of the result vector
    vecteur_x[0] = x_init

    for k in range(1,T):
        vecteur_x[k] = np.random.normal(0.5*vecteur_x[k-1] +
           25*(vecteur_x[k-1]/(1+vecteur_x[k-1]**2))+8*np.cos(1.2*k),
           np.sqrt(Q)) # fn|n-1

    return vecteur_x
```

Source 2 – Python code to generate a real trajectory from the initial position.

Exercise 1. Question 4

```
def creer_observation(vecteur_x, R):
    T = len(vecteur_x) # initialization of the result vector
    y = np.zeros(T)

    for n in range(T):
        y[n] = np.random.normal((vecteur_x[n]**2)/20, np.sqrt(R))  # gn
    return y
```

Source 3 – Python code to generate an observed trajectory.

Exercise 1. Question 6

```
def rmse(y_true, y_pred):
    return np.sqrt(np.mean((y_true - y_pred)**2))
```

Source 4 – Python code to measure the quadratic error between two trajectories.

Exercise 1. Question 5

```
def filtrage_particulaire(particules, poids, observation, Q, R, t):
    N = len(particules)
    particules_suivant, poids_suivant = np.zeros(N), np.zeros(N)
```

```python

    # prediction
    # resampling
    An = np.random.choice(particules, size=N, p=poids, replace=True)

    for j in range(N):
        # sampling
        particules_suivant[j] = np.random.normal(0.5 * An[j] + 25 * (An[j] / (1 + An[j] ** 2)) + 8 * np.cos(1.2 * t), np.sqrt(Q))

        # update
        # weight computation
        m = (particules_suivant[j] ** 2) / 20
        s = np.sqrt(R)
        poids_suivant[j] = 1/(s*np.sqrt(2*np.pi)) * np.exp(-1/2 * ((observation - m)/s)**2 )

    # normalization
    poids_suivant = poids_suivant / np.sum(poids_suivant)

    return particules_suivant, poids_suivant
```

Source 5 – Python code implementing particle filtering: obtaining particles and weights at step t+1 from the particles, weights and observation at step t.

Exercise 1. Question 7

```python
# initialization of particles, weights, estimated position at step t=-1
particules = np.zeros(N)
poids = np.ones(N) / N
est = np.zeros(T) # initialization of the estimated position

# start of filtering
t = 0 # we place ourselves at instant 0

while t != T: # the algorithm stops at step T

    # particles, weights at step t+1
    particules, poids = filtrage_particulaire(particules, poids, Y[t], Q, R, t)

    # estimated position of the target at step t
    res = 0
    for i in range(N):
        res += particules[i] * poids[i]
    est[t] = res

    # next step
    t += 1
```

Source 6 – Python code implementing particle filtering over the entire observed trajectory. We will display the curves once we have the algorithm's execution time.

Exercise 1. Question 8

```python
import time

# initialization of particles, weights, estimated position at step t=-1
particules = np.zeros(N)
poids = np.ones(N) / N
est = np.zeros(T) # initialization of the estimated position

# start of filtering
t = 0 # we place ourselves at instant 0

temps_debut = time.time()
while t != T: # the algorithm stops at step T

    # particles, weights at step t+1
    particules, poids = filtrage_particulaire(particules, poids, Y[t],
        Q, R, t)

    # estimated position of the target at step t
    res = 0
    for i in range(N):
        res += particules[i] * poids[i]
    est[t] = res

    # next step
    t += 1

temps_fin = time.time()
duree_execution = temps_fin - temps_debut
print("Program execution time:", duree_execution, "seconds")
```

Source 7 – Python code to measure the particle filtering execution time.

Exercise 1. Question 7 (continued)

```python
temps = np.arange(1, T+1)

erreur = rmse(X,est)

plt.figure()
plt.plot(temps,X, color = 'r', label = "Real trajectory")
#plt.scatter(temps,Y,marker = '+', color = 'b', linewidth =0.5, label
    = "Observed trajectory")
plt.scatter(temps,est,marker = 'x', color = 'g', linewidth =1, label =
    "Estimated trajectory")
plt.text(0.3, 0.4, f'Mean Squared Error: {erreur:.2f}',
    transform=plt.gca().transAxes, fontsize=10)
plt.text(0.01, 0.01, f'Program execution time (in seconds):
    {duree_execution:.2f}', transform=plt.gca().transAxes, fontsize=10)
plt.title("Target position over time")
plt.ylabel("Position")
plt.xlabel("Time")
plt.legend()
```

```
plt.show()
```

Source 8 – Python code to display the real, observed, and estimated trajectories. We deliberately omit the observed trajectory to have more readable curves.

Exercise 2

Several functions provided in the problem statement do not work, notably `selectionner_zone()` since no graphical window opens to select the desired points. We therefore had to adapt them, taking some liberties to complete the exercise : these therefore do not strictly follow the problem statement. In order not to excessively burden the report, we will only include here the particle filtering algorithm adapted for face tracking, which presents added mathematical value. The rest of the code is available as an attachment in `Exercice2.py`, as well as the code for exercise 1 in `Exercice1.py`.

```
def filtrage_particulaire(particules, poids, t):

    # we open the image
    image = Image.open(repertoire[t])

    particules_suivant, poids_suivant = np.zeros((2,N)), np.zeros(N) # there are N particles of dimension 2x1

    # prediction
    # resampling
    An = np.random.choice(np.arange(0, N), size=N, p=poids, replace=True)

    for j in range(N):

        # sampling
        particules_suivant[:,j] = np.random.multivariate_normal(mean=particules[:,An[j]], cov=C, size=1)[0]

        # update
        # computation of histogram q' of particules_suivant[:,j] therefore ...
        # the second argument of calcul_histogramme is a list with the x-coordinate, y-coordinate of the j-th particle, the length and width of the frame
        new_image, kmeans, histo = calcul_histogramme(image,[particules_suivant[0][j], particules_suivant[1][j], longueur, largeur], Nb)

        # Computation of D(q,q')
        D = 0
        for q in range (Nb) :
            D = D + np.sqrt(histo_ref[q]*histo[q])
        D = (1-D)**0.5

        # Weight computation and normalization
        poids_suivant[j] = np.exp(-lambda_im*(D**2))

```

```
    # Normalization
    poids_suivant = poids_suivant / np.sum(poids_suivant)

    return particules_suivant, poids_suivant
```

Source 9 – Python code implementing particle filtering: obtaining particles and weights at step t+1 from the particles, weights and observation at step t.

```
T = len(repertoire)

esperance = np.zeros((2,T)) # estimated trajectory
particules = np.random.multivariate_normal(mean=moyenne, cov=C, size =
    N)
particules = particules.T
poids = 1 / N * np.ones(N)

t=0

while t != T: # the algorithm stops at step T
    # particles, weights at step t+1
    particules, poids = filtrage_particulaire(particules, poids, t)

    # estimated position of the target at step t
    esperance[0][t] = np.sum(particules[0]*poids)
    esperance[1][t] = np.sum(particules[1]*poids)

    # next step
    print("done")
    t += 1
```

Source 10 – Python code implementing particle filtering over the entire observed trajectory.